

Case Técnico — Martech Specialist

Tempo estimado: 45 minutos a 1 hora de implementação. Não queremos que você vire noites — priorize, e se algo estiver custando mais que o previsto, documente a decisão e siga em frente.

Idioma de entrega: português ou inglês, como preferir.

Uso de IA: liberado. Usamos IA no dia a dia. Só pedimos que você entenda e saiba defender tudo o que entregar — haverá uma conversa técnica sobre suas escolhas.

Dados: os arquivos estão na pasta que acompanha este case.

1. Contexto de negócio

Você acabou de entrar no time de **Growth / CRM** da FLUTTER BRAZIL, uma operadora de apostas esportivas e cassino que atua no Brasil e recebe jogadores que depositam em **moedas diferentes** (BRL, USD e EUR).

O time vai lançar uma **campanha de reativação** para jogadores que ficaram dormentes (pararam de depositar/apostar). O orçamento é limitado, então a pergunta da liderança de Growth é direta:

"Quais jogadores dormentes vale a pena reativar, com qual oferta, e quanto eles valem para a gente?"

Hoje ninguém responde isso com confiança por dois motivos bobos, mas reais:

1. **Os valores estão em moedas diferentes** e não há uma visão unificada de valor por jogador. Um depósito de "100" pode ser R100, US100 ou €100 — e ninguém normalizou isso.
2. **A taxonomia das campanhas é uma bagunça.** O time nunca conseguiu medir com confiança qual *oferta* funciona melhor porque os nomes das campanhas não seguem o padrão.

Sua missão é construir a base de dados que resolve esses dois problemas e usá-la para recomendar **quem** a campanha deve mirar e **com qual oferta**.

Data de referência: considere "hoje" = **2024-04-01**. Toda régua temporal (ex.: dormência) deve ser calculada a partir dessa data.

2. Dados fornecidos

Os dados são fictícios e representam um ambiente real — portanto, **têm imperfeições**. Não vamos descrever as tabelas coluna a coluna: **perfilar os arquivos, entender o grão, os tipos, as chaves, como se relacionam e quais imperfeições existem é o primeiro trabalho do case.**

2a. Datasets

Na pasta data/ há **5 arquivos CSV**:

- players.csv
- deposits.csv
- bets.csv
- campaigns.csv
- campaign_touchpoints.csv

Explore-os. Documente o que descobrir sobre cada um (grão, colunas relevantes, relações) e, principalmente, **as imperfeições que encontrar e como decidiu tratá-las** — inclusive a definição de "**jogador dormente**", que você precisa definir e justificar.

2b. Padrão de taxonomia de campanha

Uma informação de negócio que você **não** teria como inferir só olhando os dados: o padrão **oficial** de nomenclatura de campanhas da Flutter Brazil (obrigatório para campanhas novas) é:

```
{geo}_{channel}_{objective}_{product}_{audience}_{period}_{offer}
```

Segmento	Vocabulário controlado
geo	BR, PT, AO
channel	email, push, sms
objective	acquisition, reactivation, retention, crosssell
product	sports, casino, both
audience	new, active, dormant, vip
period	YYYYQ# (ex.: 2024Q1)
offer	bonus50, bonus100, freebet, freespins, cashback, none

Exemplo **conforme**: `BR_email_reactivation_casino_dormant_2024Q1_bonus50`

Na prática, os nomes reais podem não seguir esse padrão. Avaliar o quanto aderem, decidir o que dá para recuperar e o que descartar, e definir a **fonte de verdade** quando houver conflito entre fontes — é com você.

2c. Fonte externa — API de câmbio

Para normalizar os valores em **BRL**, use a **Frankfurter API** (câmbio do Banco Central Europeu), **gratuita e sem autenticação**.

- **Documentação:** <https://frankfurter.dev/v1>
- **Use obrigatoriamente a versão v1 da API.** A Base URL é `https://api.frankfurter.dev/v1/` — todos os endpoints devem começar por esse prefixo (ex.: `https://api.frankfurter.dev/v1/2026-07-03..`).

3. Sua tarefa

Mais importante do que *fazer tudo* é **fazer com critério e saber explicar**. Se faltar tempo, prefira entregar menos itens bem raciocinados a entregar tudo pela metade. Em cada decisão relevante, deixe explícito **o que você assumiu, por que, e o que faria diferente com mais tempo/dados**.

3a. Engenharia de dados

1. **Ingestão robusta das cotações** da Frankfurter, com foco em uma estrutura que funcione em produção.
2. **Parse da taxonomia** de `campaign_name` em colunas estruturadas
3. **Modele os dados** brutos em tabelas limpas e prontas para análise, com todos os valores **normalizados em BRL**
4. **Desenho da solução (arquitetura):** descreva — em texto e/ou um diagrama simples — como esse pipeline **rodaria em produção**, não só uma vez no seu notebook.

3b. Análise

1. Defina, **com justificativa**, o que é um jogador **dormente** (relativo à data de referência 2024-04-01).
2. **Segmente os jogadores dormentes por valor** usando sobre os valores normalizados em BRL.
3. Mostre o **valor (LTV) por canal de aquisição** com quebras **por oferta e produto** — em BRL.

3c. Recomendação

1. Recomende **quais segmentos** a campanha deve mirar, **com qual oferta/produto**, o **valor esperado** e os **trade-offs**. Escreva essa parte para um **gerente de Growth não-técnico**.

4. Entregáveis e formato de envio

A entrega tem **duas partes**: um repositório com o trabalho e uma **apresentação ao vivo**.

4a. Repositório

Envie um **repositório Git** (link ou .zip) contendo três coisas:

- i. **Os códigos** que você usou — ingestão, transformação, análise. Como preferir organizar (scripts, notebook, SQL, dbt); só precisa rodar.
- ii. **Os arquivos finais/corrigidos** que seu código gera — as tabelas limpas e normalizadas em BRL, o relatório de conformidade da taxonomia e qualquer output que sustente a recomendação.
- iii. **O README** — que é o **entregável principal do repositório** e a documentação completa da entrega. Ele deve conter, no mínimo:
 - **Como rodar** os códigos (setup, dependências, comandos, variáveis de ambiente) — de forma que a banca consiga reproduzir.
 - **O que cada código faz** e onde ficam os arquivos finais gerados.
 - **As decisões e premissas**: o que você encontrou nos dados, como tratou cada imperfeição, como definiu "dormente", como resolveu o câmbio, como lidou com a taxonomia — e o **porquê** de cada escolha.
 - **O que ficou de fora** e o que você faria diferente com mais tempo.

4b. Apresentação ao vivo

Você fará uma sessão de **1 hora: ~30 min a 45 min de apresentação** sua + **~15 a 30 min de perguntas e discussão** com a banca.

- A apresentação é sobre **como você resolveu o problema** — o raciocínio, as decisões de arquitetura e os trade-offs. **Não é um tour pelo código linha a linha.**
- Fale como se estivesse apresentando para um time misto (pessoas técnicas e uma gerente de Growth). Termine com a **recomendação de negócio** (seção 3c): quem mirar, com qual oferta, valor esperado e por quê.
- Formato livre (slides, notebook, quadro) — o que ajudar você a **defender suas escolhas.**
- Os 30 min de perguntas cobrem tanto o que você apresentou quanto as decisões e arquitetura da solução. Esperamos que você **defenda, refine e/ou reconheça limites** das suas decisões ao vivo.

5. Regras e restrições

- Ferramentas e stack à sua escolha.
- IA liberada (veja o topo). Usamos IA no dia a dia. Só pedimos que você entenda e saiba defender tudo o que entregar — haverá uma conversa técnica sobre suas escolhas.
- **Nenhum segredo cometido.**
- **Premissas são bem-vindas** — só documente-as.
- **Fora de escopo:** dashboard/UI, deploy real, engenharia de features pesada de ML.

6. O que avaliamos (resumo)

Além da **competência e da qualidade técnica** da solução, o que mais pesa para nós é:

- **Arquitetura** — como você desenharia isso para viver em produção, não só rodar uma vez no notebook.
- **Detalhismo** — se você percebe e trata as imperfeições dos dados.
- **Qualidade da resposta final** — números consistentes, que passam num teste de sanidade, e uma recomendação clara e acionável.
- **Argumentação geral da recomendação** — como você justifica e defende as escolhas que levaram a ela.